Evaluating Policy Gradient Methods and Domain Randomization for Sim-to-Real Robot Control

Giuseppe Colazzo
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address

Francesco Federico
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address

Marco Macrì
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address

Filippo Montecchi
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address

***Abstract*—This project investigates reinforcement learning for continuous robotic control and sim-to-real transfer in a sim-to-sim setting. The work is divided into two complementary parts. In the first part, the focus is on policy-gradient methods implementation in the Hopper continuous-control environment, including REINFORCE, REINFORCE with baseline, and Actor-Critic architectures. Several improvements were introduced, including Generalized Advantage Estimation (GAE), advantage normalization, a learned exploration variance with a sigma floor, and a triggered learning-rate decay mechanism.**

The resulting REINFORCE agent with and without fixed baseline both performed similarly with an average reward of WAITING FOR MORE DATA. The resulting Actor-Critic agent successfully learned forward locomotion, achieving average rewards around 2000 and individual episodes exceeding 3000, however a significant reward variability remained throughout training.

In the second part, sim-to-sim transfer methods were deployed on the PandaPush environment, using Proximal Policy Optimization (PPO), Soft Actor-Critic (SAC), Uniform Domain Randomization (UDR), and Automatic Domain Randomization (ADR).

Experimental results showed that SAC outperformed PPO, while UDR achieved the best transfer performance among the evaluated sim-to-real strategies. Compared to vanilla SAC, UDR reduced the sim-to-real gap by approximately 66%, generating a sturdier model able to adapt to a borader range of weights, resulting in an overall better model for generalization across different environment dynamics. The results highlight the effectiveness of modern actor-critic methods for continuous control and show that domain randomization can substantially improve transfer robustness under dynamics uncertainty.

I. Introduction

PPO SAC STILL MISSING @FRANCESCO @MARCO Reinforcement Learning (RL) is becoming one of the most popular and successful approaches for solving continuous control problems, especially in robotics. (Tera et al., 2026) OR/AND (Kober et al., 2013) Indeed, this setting introduces several challenges such as delayed rewards, exploration–exploitation trade-offs, high-dimensional continuous action spaces and potentially unstable training dynamics. In addition to this, robotic control represents a challenging application of RL also due to the fact that small changes in the policy can lead to very different behaviors and delayed feedback can make credit assignment complex.

Another major challenge this paper tackles is sim-to-real transfer. Policies are commonly trained in simulation, where data collection is cheap and safe, and then applied to environments whose dynamics differ from those seen during training. This discrepancy represents the reality gap and it can significantly degrade performance. (Tobin et al., 2017) In the second half of the project, the reality gap is represented through a variation in the mass of a manipulated object: from a source environment with a 1 kg cube to a target environment with a 5 kg cube. The objective is to learn an effective control policy which is also robust to the reality gap.

The project is divided into two complementary parts. In the first part, we implement and analyze policy-gradient methods on the Hopper continuous-control environment, observing learning dynamics, exploration behavior, critic stability, and overall training performance (Brockman et al., 2016; Todorov et al., 2012). Specifically, we implement REINFORCE (Williams, 1992) with and without a constant baseline, and a one-step Actor-Critic architecture augmented with Generalized Advantage Estimation, evaluating the impact of variance reduction techniques on learning stability and convergence.

In the second part, we investigate sim-to-real transfer on the PandaPush environment [1], first evaluating an on-policy approach with Proximal Policy Optimization (PPO) [2] against Soft Actor-Critic (SAC) [3], an off-policy method, together with Uniform Domain Randomization (UDR) and Automatic Domain Randomization (ADR) [4] strategies to address the reality gap. The goal is to evaluate how algorithmic choices and domain-randomization techniques affect robustness and transfer performance across different object masses. (Gallouédec et al., 2024)

II. Methodology

A. Part 1: Hopper Continuous Control

Guidelines:

- Implementation of the different algorithms, including relevant loss functions and design choices
- Explain which advanced RL argument has been used, the selection of hyperparameters, ecc.
- Domain Randomization: Detail the implemented UDR and ADR strategies, including the chosen mass ranges, adaptation rules, and relevant hyperparameters.
- Anything else that is related to the specific project ideas and implementations Evaluation Metrics: Clearly define the metrics and protocol used to assess performance for each evaluated configuration.
- Note: you can choose whether to divide this section and the next one by part 1 and part 2 (following project specific parts) or to keep them in the same text without specific subsections. Both options are valid.

Part 1 focused on learning a locomotion policy for the Hopper environment using policy-gradient methods. Three approaches were implemented and compared: REINFORCE, REINFORCE with baseline, and Actor-Critic architectures. (Sutton & Barto, 2018)

REINFORCE (Williams, 1992) is the simplest policy algorithm, updating the policy parameters at the end of each episode using the full Monte Carlo return. The loss minimized at each update is:

$$L = -\sum_t (G_t - b) \log \pi_\theta(a_t|s_t)$$

where G_t is the discounted return from timestep t, b is a baseline and $G_t = \sum_{k=0}^{T-t} \gamma^k r_{t+k}$. This formulation ensures that the gradient estimator is unbiased for any choice of b, because $\mathbb{E}[\nabla_\theta \log \pi_\theta \cdot b] = 0$ for any constant. However, it has high variance due to the stochasticity of full-episode returns. In the first variant $b = 0$ we have vanilla REINFORCE. In the second, the return is shifted by a constant baseline $b = 20$. This helps to reduce the variance of the gradient by centering the return signal, but does not affect the direction of the gradient. Both variants share the same policy network and training configuration: a two-layer MLP with hidden sizes [64, 64] and Tanh activations, trained using the Adam optimizer with learning rate 3×10^{-4} and discount factor $\gamma = 0.99$.

We used orthogonal initialization [**?**] instead of the standard normal weight initialization for better training stability. The hidden layers used a gain of $\sqrt{2}$ to keep the gradient norms and avoid exploding or vanishing gradients. Furthermore, the output layer of the actor was initialized with a low gain of 0.01. This makes sure the initial policy outputs action values close to zero, stopping extreme movements that would destabilize the Hopper right away. Therefore, the early exploration is entirely governed by the learned standard deviation, which allows the agent to survive for a longer period of time and collect better trajectories during the initial training stage [**?**].

The final implementation uses an Actor-Critic architecture, where the actor learns the policy while the critic estimates the value function. The actor updates the policy based on the estimated advantages, modifying the probability of actions based on how they turned out to be with respect to the critic's expectations:

$$L_{\text{actor}} = -\mathbb{E}\left[\hat{A}_t \log \pi_\theta(a_t|s_t)\right].$$

In parallel, the critic minimizes the squared difference between the predicted state value and the temporal-difference target:

$$L_{\text{critic}} = \mathbb{E}\left[\left(r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)\right)^2\right].$$

Since we're considering a continuous enviroment, all models policy outputs both the mean and standard deviations of a Gaussian action distribution:

$$a_t \sim \mathcal{N}(\mu_\theta(s_t), \sigma_\theta(s_t)),$$

where both parameters are learned by the policy network. During training, actions are sampled from the distribution to encourage exploration, while during evaluation the policy becomes deterministic by using mean actions $\mu_\theta(s_t)$.

To improve the quality of the policy updates, Generalized Advantage Estimation (GAE) was implemented. (Schulman et al., 2015) Therefore, advantages are computed as:

$$\hat{A}_t^{GAE} = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}$$

Where:

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

is the temporal-difference error.

In this way, GAE provides a compromise between using a low-variance TD(0) estimate and a high variance Monte Carlo approach that considers the episode full return.

The final Actor-Critic configuration employed a discount factor $\gamma = 0.99$, a GAE parameter $\lambda = 0.95$, a learned action variance, and a minimum variance threshold (sigma floor = 0.1). Training was performed for more than 70,000 episodes in order to allow the policy sufficient time to discover and refine locomotion behaviours. These hyperparameters were selected empirically to balance exploration, learning stability, and long-term performance.

Subsequently, advantages were normalized before policy updates in order to stabilize gradients, stabilize critic's estimates and reduce training instability. A minimum standard deviation (sigma floor) was enforced to prevent premature collapse of exploration and continue promoting exploration even in later stages. Additionally, a triggered learning-rate decay mechanism was introduced during later stages of training with the aim of stabilizing training by reducing update magnitude after the policy had discovered effective locomotion behaviors.

TABLE I
ACTOR-CRITIC HYPERPARAMETERS

Hyperparameter	Value
Discount Factor (γ)	0.99
GAE Lambda (λ)	0.95
Learning Rate	3×10^{-4}
Minimum Learning Rate After Decay	1×10^{-5}
Sigma Floor	0.1
Initial Learned Sigma	0.5 (through Softplus + Sigma Floor)
Entropy Coefficient	0.0
Training Episodes	120,000
Learning Rate Decay Trigger	∼45,000 episodes (performance-triggered)
Hidden Layers	64, 64
Activation Function	Tanh
Optimizer	Adam

More specifically, the decay was activated only once one of the following trigger conditions was satisfied, which caused the learning rate to linearly reduce from 3×10^{-4} to 10^{-5} until the end of training.

TABLE II
LEARNING RATE DECAY TRIGGERS

Trigger condition	Threshold
Rolling average reward and rolling average episode length	avg_reward_100 $\geq$ 1000 and avg_length_100 $\geq$ 500
Best rolling average reward	best_avg_reward $\geq$ 1500
Episode fallback trigger	episode $\geq$ 45,000
Initial learning rate	3×10^{-4}
Minimum learning rate	10^{-5}

The goal was to reduce update magnitude once the Hopper had likely discovered jumping, as this behaviour leads to higher rewards but while being much more unstable. The thresholds were chosen empirically: high average reward together with long episode length meant sustained hopping was disovered, and also best_avg_reward $\geq$ 1500 indicated clear high-performance runs. The 45,000-episode condition acted as a fallback that would have forced the LR decay initialisation, again this threshold value was set based on previous runs, which showed that hopping usually emerged around 40k–45k episodes.

All these modifications were designed to improve training stability, maintain sufficient exploration, and allow longer training horizons without facing policy collapses.

Performance was evaluated using numeruous training diagnostics, such as episodic reward, rolling average reward over the last 100 episodes, episode length, policy entropy, learned action standard deviation, actor loss and critic loss. During the training phase all the mentioned evaluation metrics were monitored to supervise the learning progress, exploration behaviour, value-function stability, and policy convergence.

B. Part 2: PandaPush, PPO, SAC and Domain Randomization

Idea:

- Not sure about anything I'm saying, check pls + it's mainly AI generated

Part 2 investigates sim-to-real transfer using the PandaPush-v3 robotic manipulation environment. The task consists of controlling a Franka Panda robotic arm to push a cube toward a desired target position. To move the arm, the chosen algorithm has control over the 7 joints of the robot, with the reward depending only on end-effector movements. Therefore, the agent must learn to coordinate joint movements in order to move the cube through the arm's end-effector and achieve task success. The observation space contains the robot state, cube state, and target position information. Actions are continuous Cartesian end-effector displacements, **need to double check this** defined as:

$$a_t = [dx, dy, dz].$$

The dense reward function is the euclidean distance between the cube and target positions, defined as

$$r_t = - \left\| p_{obj}(t) - p_{goal} \right\|_2 ,$$

where p_{obj} and p_{goal} denote the cube and target positions respectively. A task is considered successful when the final cube-target distance is below 5 cm.

Two reinforcement-learning algorithms were evaluated: Proximal Policy Optimization (PPO) and Soft Actor-Critic (SAC). Proximal Policy Optimization (PPO) is an on-policy actor-critic algorithm that limits policy updates through a clipped objective.

$$L^{CLIP} = \mathbb{E}\Big[\min(\rho_t \hat{A}_t, \text{clip}(\rho_t, 1-\epsilon, 1+\epsilon)\hat{A}_t) \Big],$$

where $\hat{A}_t$ is the advantage estimate, ϵ is a hyperparameter that controls the clipping range, and

$$\rho_t = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$$

is the probability ratio between the new and old policies.

The idea behind this algorithm is that excessively large policy updates result in unstable training, so stability is achieved to keep policy updates within a trust region (PPO is in fact the direct evolution of the Trust Region Policy Optimization (TRPO) [5] algorithm). The clipping mechanism improves stability but reduces sample efficiency because collected trajectories cannot be reused extensively.

Soft Actor-Critic (SAC) is an off-policy actor-critic method that maximizes both reward and policy entropy,

$$J = \mathbb{E}[R] + \alpha H(\pi).$$

SAC combines a replay buffer, twin critics to reduce value overestimation, and automatic entropy tuning. These characteristics generally provide better exploration and sample efficiency than PPO, especially in continuous-control robotic tasks. To improve transfer across different dynamics, Domain Randomization (DR) was applied by varying the cube mass during training.

Three configurations were evaluated:

- No DR: fixed source mass $m = 1$ kg.
- Uniform Domain Randomization (UDR): $m \sim U(0.5, 8.0)$ with a new mass sampled at every episode. **Marco should take a better look at the range to write here**

- Automatic Domain Randomization (ADR): an adaptive curriculum that expands or contracts the sampling range according to agent performance.

The target environment uses a cube mass of 5 kg. However, the randomization range was chosen to be broad enough to include both source and target conditions without being narrowly centered around the target mass, avoiding information leakage from the evaluation environment.

III. Experiments & Results

A. Part 1 Results: Hopper Continuous Control

Guidelines:

- MANDATORY The Results Table: Present the mandatory results table clearly, including all required training→test configurations.
  - **MISSING**
- Quantitative Breakdown: Describe the numbers objectively.
  - Ok
- Point out where the policies performed well
  - Ok
- Point out where they failed
  - Ok
- Tables and figures can (and should) be inserted to make the results clear to the reader.
  - **MISSING**
- Example 1 from guidelines: reinforce, reinforce + baseline and actor critic could be compared using a plot with mean and std over time to show the specific features/behaviour of the algorithms ⇒ Comparison across algorithms.
  - **MISSING: We don't have something like this**

Note:

- Results using 120k steps or 50k? (I'll use the 12k steps because it's the one I know the results best)
- I've pasted a few graphs but do we won't use all of them, select whatever you want

We ran the Actor-Critic for 70,000 training episodes and it successfully learned a locomotion policy for the Hopper environment. As we can see from the graph X (there should be the graph number), training can be divided into four distinct phases.

During the first phase (0–40k episodes), the agent learned balancing and standing behaviors, but not forward locomotion yet. Therefore, average rewards remained relatively low and episode lengths were short.

Between 40k and 60k episodes, a steep increase in both reward and episode length occurred, suggesting that forward locomotion was discovered.

Then, from 60k to 80k episodes, performance continued to improve as the policy refined its locomotion strategy.

Finally, during the 80k–120k interval, training entered a high-performance plateau without managing to converge in a stable way. Average rewards exceeded 2000 and several individual episodes achieved rewards over 3000. Nevertheless, a noticeable amount of variability still persisted in the later training stages, alternating both extremely high and low performing episodes proving that the learned behavior was able to achieve high score but not consistently.

During the whole training phase, both the learned policy standard deviation and entropy decreased throughout training (metto grafico anche di questi? Troppo? Io metterei uno dei due), showing a smooth transition from an exploratory to a more deterministic policy as it gained confidence.

The critic loss, however, present several large spikes, particularly during periods of rapid policy improvement. This is due to the fact that the critic struggles to adapt its value estimates to the rapidly evolving policy. On the contrary, the actor loss remains stably bounded and centered around zero thanks to advantage normalization.

Although convergence and behavior reproducibility was not fully achieved, the agent consistently maintained high reward levels throughout the final stages of training.

B. Part 2 Results: PandaPush Sim-to-Sim Transfer

Idea:

- Show results of each algorithm ⇒ SAC + UDR → SAC + ADR → SAC → PPO
- Reality gap ⇒ vanilla SAC vs upper bound ; SAC + UDR vs upper bound
- Mass-sensitivity analysis ⇒ vanilla SAC sucks and SAC + UDR achieves better results for general purpose
- Again, not sure about the correctness and pretty AI

(Marco ha messo una tabella in report insights pt2 ma è gigantesca, non so se ci stia + non immediata da leggere), inserire valore corretto per PPO e aggiungere success rate

TABLE III
Target-environment performance comparison.

Method	Target Return
PPO	~ -3.0
SAC (none)	-0.633
SAC + ADR	-0.556
SAC + UDR	-0.509
Oracle	-0.445

The results clearly show that SAC outperforms PPO. Even after an extensive hyperparameter tuning, PPO consistently plateaued around a return of -3.0 and failed to solve the task consistently (success rate:$\sim 20\%$). On the other hand, SAC successfully learned effective pushing policies and transferred well to the target environment.

Uniform Domain Randomization (UDR) achieved the best performance, outperforming both vanilla SAC and Automatic Domain Randomization (ADR). It achieved a target return of -0.509 compared to -0.556 for ADR and -0.633 for vanilla SAC. The policy obtained training directly on the target environment (oracle) achieved the best overall result (-0.445) but is not a valid sim-to-real solution because it has access to target dynamics during training, it only represents the upper bound.

The reality gap can be measured relative to the oracle.

- The gap between vanilla SAC and the oracle is $(-0.445) - (-0.633) = 0.188$.
- The gap between SAC + UDR and the oracle is $(-0.445) - (-0.509) = 0.064$.

Therefore, the fraction of the gap removed by UDR is

$$\frac{0.188 - 0.064}{0.188} = 0.6596,$$

corresponding to approximately 66%.

This result indicates that UDR eliminates about two-thirds of the performance loss caused by the source-to-target mass shift.

Finally a mass-sensitivity analysis was conducted to further highlight the robustness gained thanks to domain randomization techniques. When evaluated at increasingly heavier masses, vanilla SAC deteriorates from -0.554 at 1 kg to -0.869 at 15 kg. In contrast, UDR only decreases from -0.478 to -0.592 over the same range. This demonstrates that UDR learns a policy that is robust across a wide range of dynamics rather than specialized to a single mass value.

IV. Discussion

Guidelines:

- Part 1:
  - Result Analysis: Go beyond the numbers and explain why the results happened: ok
  - Methodological Critique: Reflect on the limitations of the approaches: ok
  - e.g., algorithm stability, hyperparameter sensitivity, UDR/ADR limitations, and the simplified sim-to-sim setting.
    - algorithm stability: ok
    - Hyperparameter sensitivity: meh, not really but I think it's fine, the guidelines cite the hyperparameter sensitivity for the methodological critique, but we criticize more than enough the model, so there might be no need to add hyperparameter sensitivity.
- Part 2:
  - AI generated

The results obtained in the first part demonstrate that the proposed Actor-Critic framework successfully learned a locomotion policy for the Hopper environment. Indeed, the Hopper's discovered balancing, the forward locomotion behavior, the average rewards exceeding 2000 together with the occasional episodes reaching scores above 3000, all prove that the agent discovered effective hopping. Nevertheless, training did not fully converge to a stable hopping policy: significant reward variance remained throughout the final stages of training, proving that the learned policy was still too exploratory and stochastic, causing it to be unable to reproduce its best behaviors consistently.

This result represents a substantial improvement compared to earlier Actor-Critic implementations tested during development. Preliminary versions frequently converged to a degenerate local optimum in which the Hopper remained stationary for extended periods before eventually collapsing. Although such behaviour produced relatively stable trajectories, it prevented the discovery of effective locomotion and resulted in poor returns. The introduction of GAE, advantage normalization, a learned exploration variance with a sigma floor and a triggered learning-rate decay mechanism significantly improved exploration and enabled the policy to free itself from the local optimum, eventually discovering hopping.

This behavior instability seems to be proved by the critic-loss spikes, which, however should not necessarily be interpreted as training failure. These spikes are not necessarily indicative of unstable training as reinforcement-learning targets continuously change together with the current policy, therefore when the actor discovers substantially better trajectories, previously learned value estimates become obsolete and inaccurate, generating large temporal-difference errors and consequently large critic-loss spikes. This reasoning proves that the critic loss irregularity is caused by its own outdated estimates and it trying to keep up with the new updated policy, demonstrating that an unsteady critic loss is quite inevitable as it is caused by the Reinforcement Learning intrinsic dynamicity.

This interpretation is supported by the fact that critic spikes often coincide with newly discovered high-reward behaviors, such as the discovery of hopping.

In contrast, actor updates remained remarkably stable throughout training, as advantage normalization successfully bounded policy updates and prevented gradient explosions despite the highly non-stationary learning process.

Despite these improvements, the final policy still exhibits substantial variability and has not fully converged. Future work could investigate additional stabilization techniques like TRPO or PPO which exploits trust region and clipped policy updates (Schulman et al., 2017).

Furthermore, more advanced actor-critic methods for continuous control, such as DDPG and TD3, could be explored, exploiting double-critic architectures and delayed policy updates, to potentially reduce value-estimation errors and improve training stability (Fujimoto et al., 2018).

V. Conclusion

Guidelines:

- best-performing algorithm: ok
- effectiveness of domain randomization: ok
- main limitation or possible future improvement: ok

This project investigated reinforcement learning for continuous control and sim-to-real transfer through two complementary tasks.

In Part 1, the REINFORCE (with and without baseline) and a custom Actor-Critic framework was developed and all were applied to the Hopper environment. The proposed enhancements, including Generalized Advantage Estimation (GAE), advantage normalization, a learned policy standard

deviation, a sigma floor, and triggered learning-rate decay, allowed to achieve an average rward of over 2000 and single episodes well over 3000, proving that they achieved successful learning of locomotion behaviours. Nevertheless, significant reward variability and the lack of behavioral convergence persisted throughout training, indicating that the agent was not able to always replicate consistent hopping.

In Part 2, the focus shifted to sim-to-real transfer in the PandaPush environment. The experimental results demonstrated a clear advantage of Soft Actor-Critic (SAC) over Proximal Policy Optimization (PPO), which failed to solve the task within the available training budget. Among the transfer strategies deployed to fill the reality gap, Uniform Domain Randomization (UDR) achieved the best overall performance, outperforming both vanilla SAC and unexpectedly Automatic Domain Randomization (ADR) too.

The final selected approach is therefore SAC combined with UDR. By training on a broad distribution of cube masses, UDR produced a substantially sturdier policy and reduced the sim-to-real gap by approximately 66% relative to the non-randomized baseline. These results confirm the effectiveness of domain randomization as a practical strategy for improving transfer performance in robotic reinforcement-learning tasks.

References

- (Tera, K., Adetifa, A., & Udekwe, D. (2026, March 15). Taxonomy and trends in reinforcement learning for robotics and control systems: A structured review. arXiv. https://arxiv.org/abs/2510.21758)
- Kober, J., Bagnell, J. A., & Peters, J. (2013, September). Reinforcement learning in robotics: A survey. The International Journal of Robotics Research, 32(11), 1238–1274. https://doi.org/10.1177/0278364913495721
- Tobin, J., Fong, R., Ray, A., Schneider, J., Zaremba, W., & Abbeel, P. (2017, March 20). Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World. arXiv.org. https://arxiv.org/abs/1703.06907
- Brockman, G., Cheung, V., Pettersson, L., Schneider, J., Schulman, J., Tang, J., & Zaremba, W. (2016). OpenAI Gym. arXiv. https://arxiv.org/abs/1606.01540
- Todorov, E., Erez, T., & Tassa, Y. (2012). MuJoCo: A physics engine for model-based control. In 2012 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS) (pp. 5026–5033). IEEE. https://doi.org/10.1109/IROS.2012.6386109
- Gallouédec, Q., Cazin, N., Dellandréa, E., & Chen, L. (2024). Gymnasium Robotics. GitHub repository. https://github.com/Farama-Foundation/Gymnasium-Robotics
- Sutton, R. S., & Barto, A. G. (2018). Reinforcement learning: An introduction (2nd ed.). MIT Press. http://incompleteideas.net/book/the-book-2nd.html
- Schulman, J., Moritz, P., Levine, S., Jordan, M., & Abbeel, P. (2015, June 8). High-Dimensional continuous control using generalized advantage estimation. arXiv.org. https://arxiv.org/abs/1506.02438
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017, July 20). Proximal Policy optimization Algorithms. arXiv.org. https://arxiv.org/abs/1707.06347
- Fujimoto, S., Herke, V. H., & Meger, D. (2018, February 26). Addressing function approximation error in Actor-Critic methods. arXiv.org. https://arxiv.org/abs/1802.09477
- Saxe, A. M., McClelland, J. L., & Ganguli, S. (2013, December 19). Exact solutions to the nonlinear dynamics of learning in deep linear neural networks. arXiv. https://arxiv.org/abs/1312.6120
- Andrychowicz, M., et al. (2020, June 18). What matters in on-policy reinforcement learning? A large-scale empirical study. arXiv. https://arxiv.org/abs/2006.05990

More stuff not cited but which might need to be:

- PPO;
- SAC;
- sim-to-real/domain randomization;
- Stable-Baselines3;

References

[1] Q. Gallouédec, N. Cazin, E. Dellandréa, and L. Chen, "panda-gym: Open-Source Goal-Conditioned Environments for Robotic Learning," *4th Robot Learning Workshop: Self-Supervised and Lifelong Learning at NeurIPS*, 2021.

[2] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *CoRR*, vol. abs/1707.06347, 2017. [Online]. Available: http://arxiv.org/abs/1707.06347

[3] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," *CoRR*, vol. abs/1801.01290, 2018. [Online]. Available: http://arxiv.org/abs/1801.01290

[4] OpenAI, I. Akkaya, M. Andrychowicz, M. Chociej, M. Litwin, B. McGrew, A. Petron, A. Paino, M. Plappert, G. Powell, R. Ribas, J. Schneider, N. Tezak, J. Tworek, P. Welinder, L. Weng, Q. Yuan, W. Zaremba, and L. Zhang, "Solving rubik's cube with a robot hand," *CoRR*, vol. abs/1910.07113, 2019. [Online]. Available: http://arxiv.org/abs/1910.07113

[5] J. Schulman, S. Levine, P. Moritz, M. I. Jordan, and P. Abbeel, "Trust region policy optimization," *CoRR*, vol. abs/1502.05477, 2015. [Online]. Available: http://arxiv.org/abs/1502.05477